

PHYS 4027 / 5027 • FALL 2026 • LECTURE 10

Multistep Methods and Energy Conservation



Prof. Alex Chen

Department of Physics

Washington University in St. Louis

The last two weeks



	the question	the answer
Lecture 7	How is a Runge-Kutta method built?	A Butcher tableau, with coefficients fixed by matching the Taylor expansion
Lecture 7	Why does RK4 blow up on a slow solution?	Stiffness: the fast timescale limits h through stability; implicit methods remove the limit
Lecture 8	How large should each step be?	Estimate the error of every step and choose h to meet a tolerance; Dormand-Prince gets the estimate almost for free
Lecture 9	What about output between steps, or ending on an event?	Dense output, and root finding on the interpolant

Part I · Multistep methods



Reusing the past

A Runge-Kutta step forgets



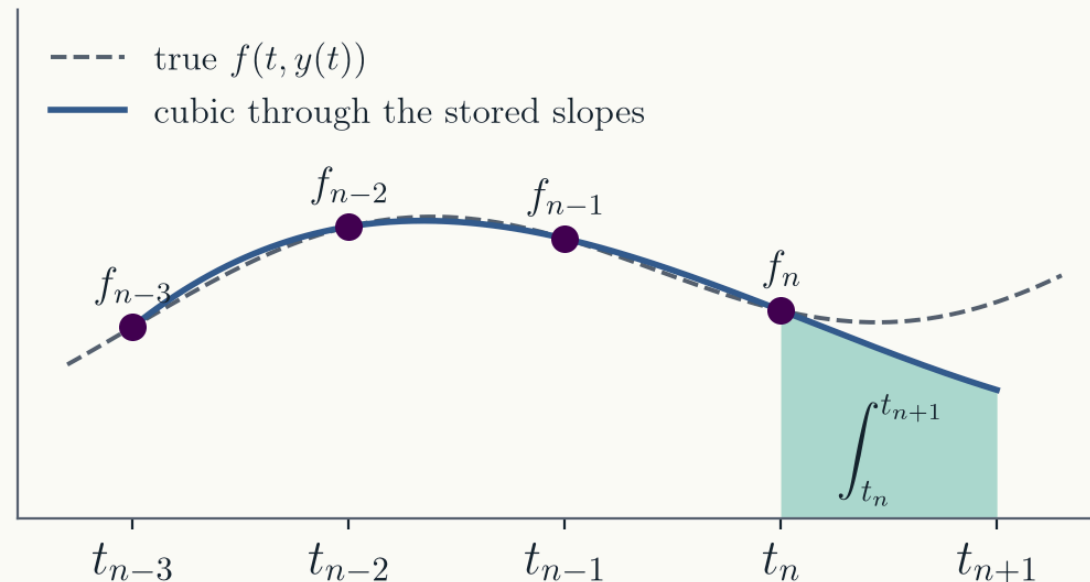
- RK4 evaluates f four times per step, combines the slopes, and throws them away. The next step starts from y_{n+1} alone.
- But the slopes of earlier steps, $f_{n-1} = f(t_{n-1}, y_{n-1})$, f_{n-2} , and so on, are good information about how f is changing.
- A **multistep method** keeps the last few slopes and builds y_{n+1} from them. Each step then needs only one new evaluation, f_n .

Runge-Kutta methods are called one-step methods: y_{n+1} depends on y_n only.

Where the coefficients come from

Integrate the ODE over one step, then replace f by the polynomial through the stored slopes:

$$y_{n+1} = y_n + \int_{t_n}^{t_{n+1}} f(t, y(t)) dt$$



The Adams-Bashforth methods

$$\text{AB2: } y_{n+1} = y_n + \frac{h}{2}(3f_n - f_{n-1})$$

$$\text{AB3: } y_{n+1} = y_n + \frac{h}{12}(23f_n - 16f_{n-1} + 5f_{n-2})$$

$$\text{AB4: } y_{n+1} = y_n + \frac{h}{24}(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3})$$

- k stored slopes give order k . AB1 is Euler's method.
- Explicit, and one new evaluation of f per step, whatever the order.

Adams-Moulton: include the new point



Let the interpolating polynomial pass through f_{n+1} as well, which interpolates instead of extrapolating:

$$\text{AM4: } y_{n+1} = y_n + \frac{h}{24} (9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2})$$

- More accurate at the same order: the local error is $\frac{19}{720} h^5 y^{(5)}$, against $\frac{251}{720} h^5 y^{(5)}$ for AB4.
- But $f_{n+1} = f(t_{n+1}, y_{n+1})$ contains the unknown: AM4 is implicit, like implicit Euler in Lecture 7.
- Implicit does not mean unconditionally stable: AM4 is stable only for $h\lambda \geq -3$ on the real axis. No multistep method of order above 2 is stable for every $h\lambda$ in the left half plane (Dahlquist).

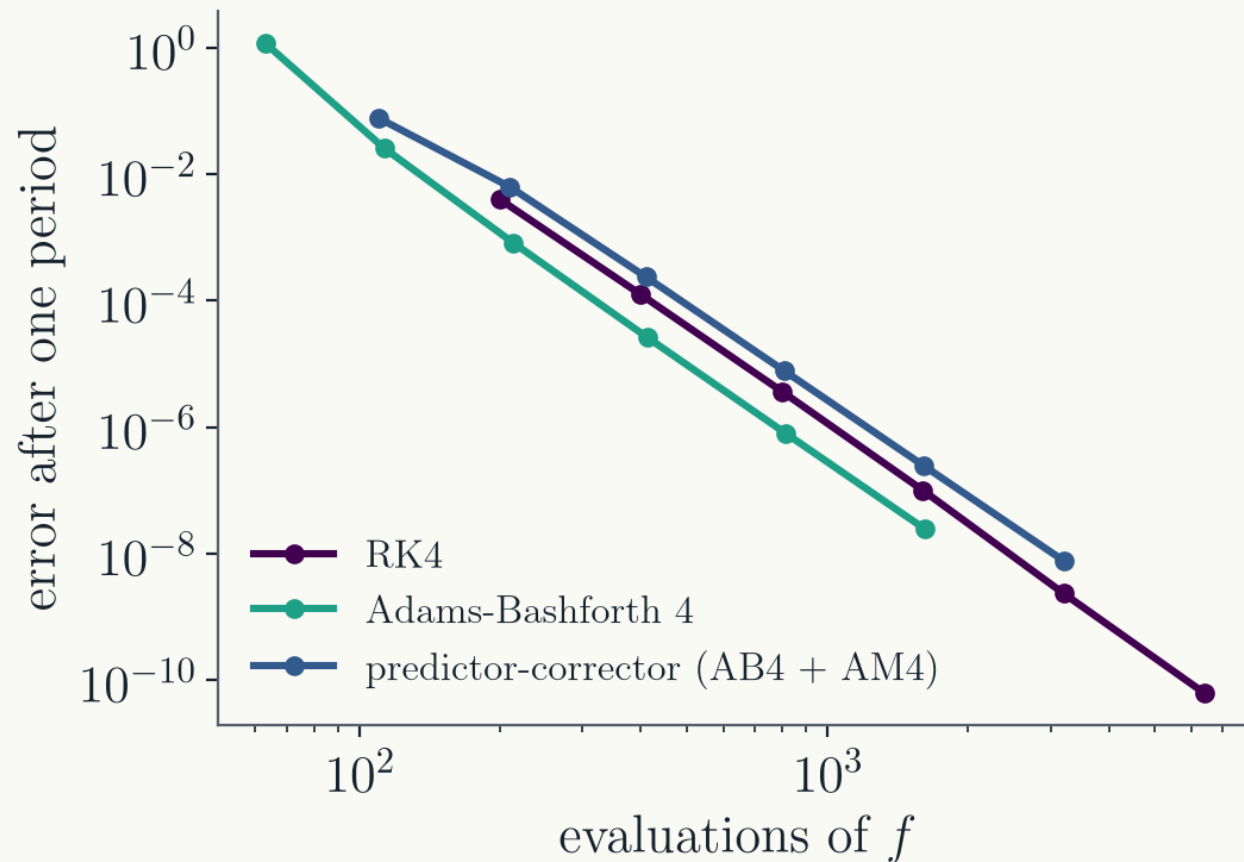
Predictor-corrector



Avoid solving for y_{n+1} : guess it explicitly, then use the guess in the implicit formula.

1. Predict $\tilde{y}_{n+1}$ with AB4.
 2. Evaluate $\tilde{f}_{n+1} = f(t_{n+1}, \tilde{y}_{n+1})$.
 3. Correct: AM4 with $\tilde{f}_{n+1}$ in place of f_{n+1} .
 4. Evaluate f_{n+1} at the corrected y_{n+1} , for use in later steps.
- "PECE": two evaluations per step.
 - $|y_{n+1} - \tilde{y}_{n+1}|$ estimates the local error for free, like the embedded pair in Dormand-Prince (Lecture 8).

Cost vs accuracy



The 170° pendulum over one period, fixed h . At about 400 evaluations of f :

RK4	1.2×10^{-4}
AB4	2.6×10^{-5}
AB4 + AM4	2.4×10^{-4}

All three are fourth order. Per evaluation, AB4 wins by a factor of about 4 here.

The corrector costs a second evaluation per step; what it buys is the error estimate and better stability, not accuracy per evaluation.

The catch



- **Start-up.** AB4 needs three earlier slopes that do not exist at t_0 . The first steps are taken with a one-step method, such as RK4.
- **Fixed step.** The weights assume equally spaced slopes. Changing h means re-interpolating the history, so step control is harder than in Dormand-Prince.
- **Stability shrinks with order.** Largest stable $|h\lambda|$ on the negative real axis:

AB1	AB2	AB3	AB4	RK4
2	1	0.55	0.30	2.79

Even per evaluation, RK4 ($2.79/4 = 0.70$) is more stable than AB4.

Backward differentiation formulas



For stiff problems, turn the idea around: interpolate the stored *values* $y_{n+1}, y_n, y_{n-1}, \dots$, and require the polynomial's slope at t_{n+1} to equal f_{n+1} .

$$\text{BDF1: } y_{n+1} - y_n = h f_{n+1}$$

$$\text{BDF2: } y_{n+1} - \frac{4}{3}y_n + \frac{1}{3}y_{n-1} = \frac{2}{3}h f_{n+1}$$

- BDF1 is implicit Euler, from Homework 4.
- Implicit and stable far into the left half plane: the standard stiff solvers (scipy's `solve_ivp(method="BDF")`, CVODE) use BDF of orders 1 to 5, changing order and step as they go.

Part II · Energy conservation



Long runs

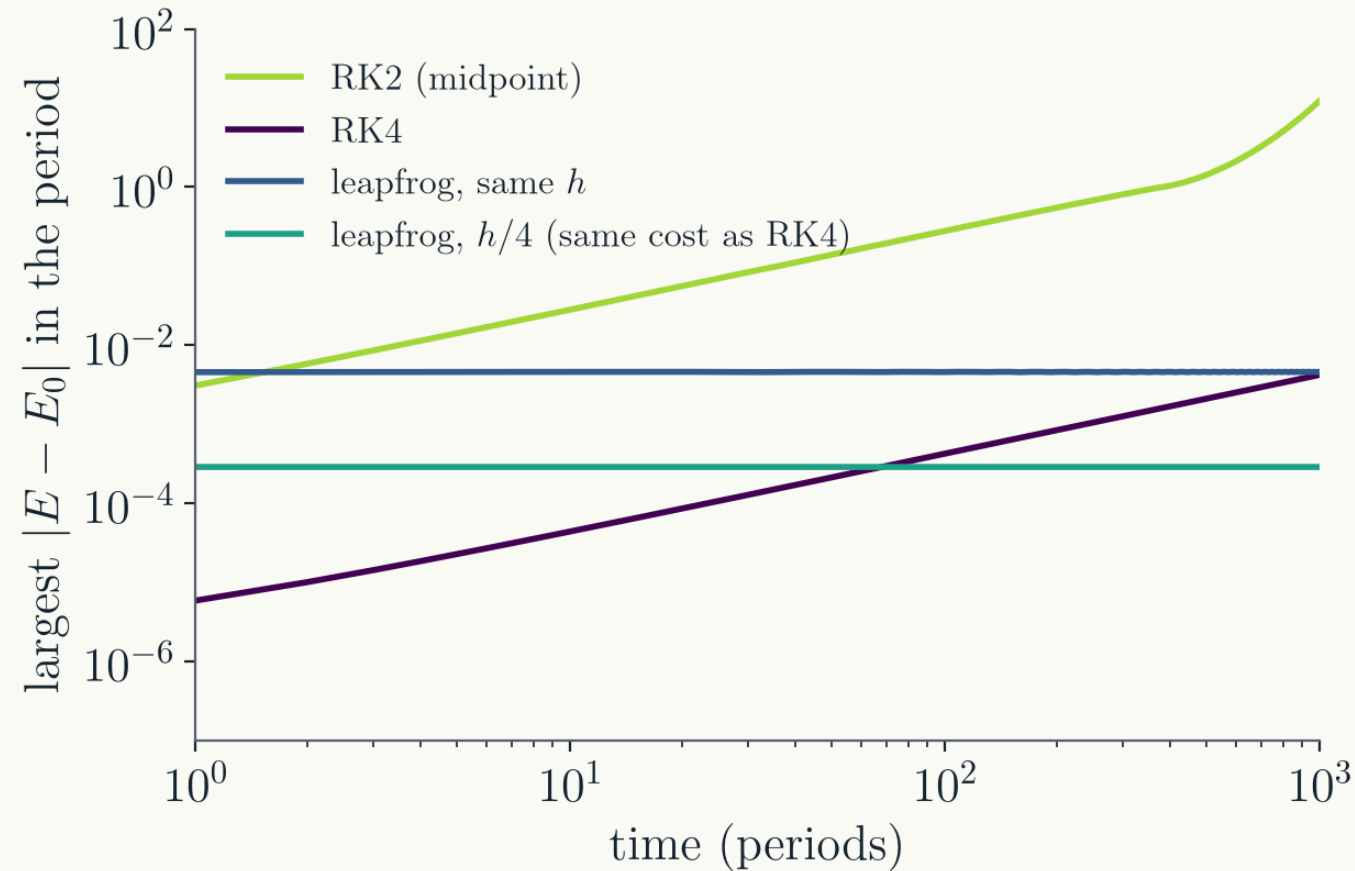
Small errors, many periods



- Homework 4: Euler pushed the pendulum over the top within one swing, while RK4 lost only 2.5×10^{-11} of energy in two periods.
- Many simulations run for far longer than two periods: planetary orbits over billions of revolutions, molecular dynamics over millions of steps.
- A tiny energy error per period, repeated a million times, is no longer tiny.

The question is not only how large the error is after one period, but whether it keeps growing.

1000 periods of a pendulum

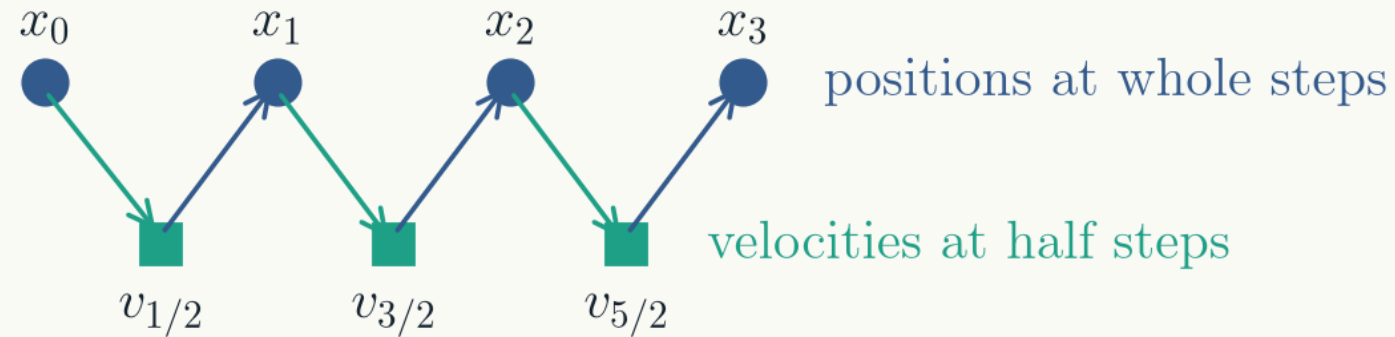


$$\theta_0 = 90^\circ, h = T/50.$$

- **RK2** gains energy until the pendulum spins over the top.
- **RK4** loses about 4×10^{-6} per period, steadily: 4×10^{-3} after 1000.
- **Leapfrog** (next slides) has a larger error at first, which never grows.
- At the same cost as RK4, leapfrog is better after about 70 periods, and the gap widens forever.

From midpoint to leapfrog

The midpoint method (Lecture 6) computes a value at the half step, uses it once, and throws it away. Keep it instead, and let positions and velocities alternate:



$$v_{n+1/2} = v_{n-1/2} + h a(x_n), \quad x_{n+1} = x_n + h v_{n+1/2}$$

Each update uses the other variable at the midpoint of its own step: second order, and one evaluation of the force $a(x)$ per step.

Velocity Verlet



For $\ddot{x} = a(x)$, the same method with positions and velocities reported at the same times: half a kick, a drift, half a kick.

$$v_{n+1/2} = v_n + \frac{h}{2} a(x_n)$$

$$x_{n+1} = x_n + h v_{n+1/2}$$

$$v_{n+1} = v_{n+1/2} + \frac{h}{2} a(x_{n+1})$$

- $a(x_{n+1})$ is reused in the first half kick of the next step: still one evaluation per step.
- This is the workhorse of molecular dynamics, where it is called velocity Verlet.

Leapfrog runs backwards exactly



- Every substep is undone by the same substep with $-h$: $x_{n+1} = x_n + h v$ is reversed by $x_n = x_{n+1} - h v$, with the same v .
- So reversing the velocity and running the same number of steps retraces the trajectory, as the exact dynamics does.
- Measured at $\theta_0 = 90^\circ$, $h = 0.1$: run 1000 steps, flip v , run 1000 more.

distance from the start	RK4	2.1×10^{-4}
	leapfrog	6.3×10^{-14}

Leapfrog's only error on the way back is rounding.

Hamiltonian structure



The pendulum, like every conservative mechanical system, is a Hamiltonian system:

$$H(q, p) = \frac{p^2}{2} + V(q), \quad \dot{q} = \frac{\partial H}{\partial p} = p, \quad \dot{p} = -\frac{\partial H}{\partial q} = -V'(q)$$

with $V(q) = -\cos q$ for the pendulum. Two consequences:

- **Energy is conserved:** $\dot{H} = \frac{\partial H}{\partial q} \dot{q} + \frac{\partial H}{\partial p} \dot{p} = \frac{\partial H}{\partial q} \frac{\partial H}{\partial p} - \frac{\partial H}{\partial p} \frac{\partial H}{\partial q} = 0.$
- **Phase-space area is conserved** (Liouville's theorem): the velocity field $(\dot{q}, \dot{p})$ has zero divergence, $\frac{\partial \dot{q}}{\partial q} + \frac{\partial \dot{p}}{\partial p} = \frac{\partial^2 H}{\partial q \partial p} - \frac{\partial^2 H}{\partial p \partial q} = 0.$

A numerical method can try to keep either property. Leapfrog keeps the second one exactly.

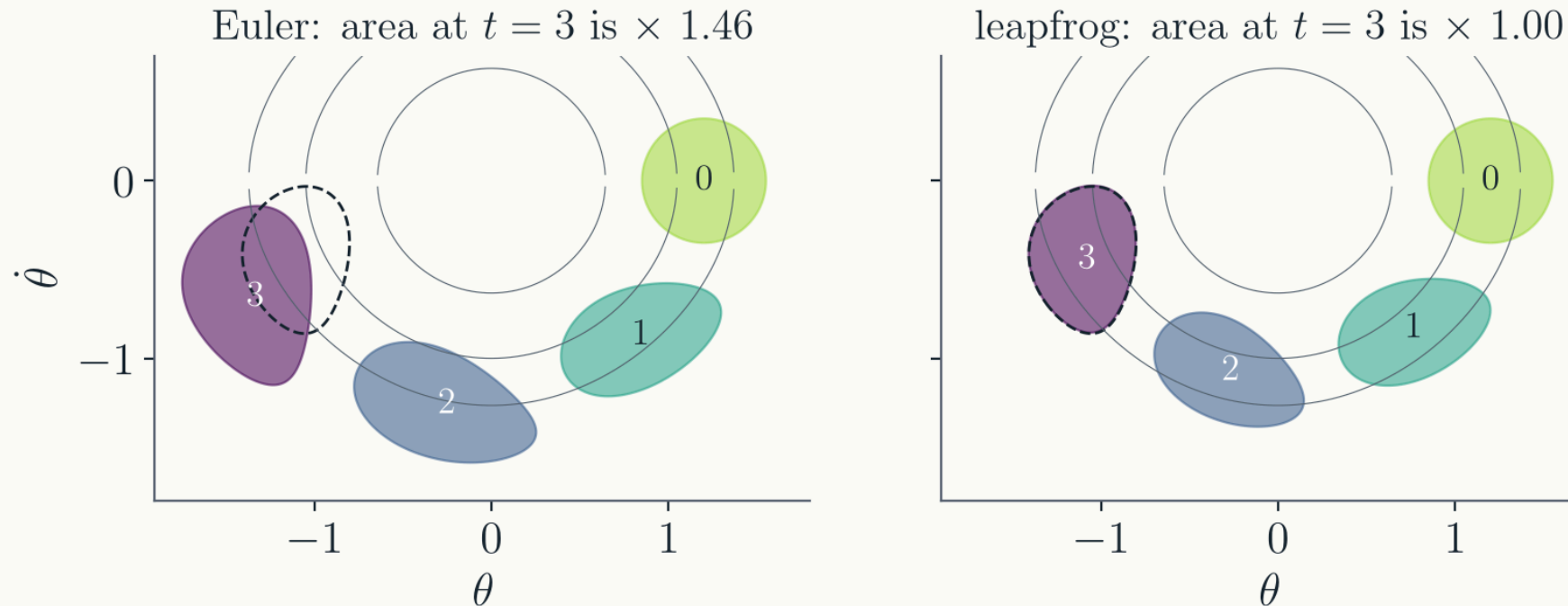
Leapfrog is built from exact solutions

Split $H = T(p) + V(q)$, with $T = p^2/2$, and solve each half on its own:

Hamiltonian	equations	exact solution for a time τ
$V(q)$ alone	$\dot{q} = 0, \dot{p} = -V'(q)$	$p \rightarrow p - \tau V'(q)$: a kick
$T(p)$ alone	$\dot{q} = p, \dot{p} = 0$	$q \rightarrow q + \tau p$: a drift

- A Verlet step is the exact flow of V for $h/2$, then of T for h , then of V for $h/2$.
- Each piece is an exact Hamiltonian flow, so each preserves phase-space area, and so does the whole step.
- Euler moves q and p at once, both from the old state: it is not the exact flow of anything.

Phase-space area



- A disk of initial conditions for the pendulum at $t = 0, 1, 2, 3$, with $h = 0.2$. Dashed: the exact patch at $t = 3$.
- Leapfrog keeps the area exactly while the patch stretches. Euler's patch grows: $\times 1.09, 1.28, 1.46$.
- Methods that preserve this structure are called **symplectic**.

Area and energy on the harmonic oscillator

On $\ddot{x} = -x$ every method here multiplies (x, v) by a matrix M , and the energy curves $x^2 + v^2 = 2E$ are circles. A Runge-Kutta step rotates and rescales, so it multiplies the area *and* the energy by $\det M$, every step:

	Euler	RK2	RK4	leapfrog
$\det M$ per step	$1 + h^2$	$1 + h^4/4$	$1 - h^6/72 + \dots$	1
energy	grows	grows	decays	bounded

- For Runge-Kutta, the energy error *is* the area error: $\det M = |R(ih)|^2$, with R the amplification factor of Lecture 7, close to 1 but never equal to it.
- Leapfrog's M has determinant exactly 1 but is not a rotation: it carries circles to ellipses of the same area. E wobbles; the shadow energy $\frac{1}{2}v^2 + \frac{1}{2}\left(1 - \frac{h^2}{4}\right)x^2$ is conserved exactly.

Why the energy error stays bounded



- Kicks and drifts do not commute, so a Verlet step is not the exact flow of H . But it is the exact flow of a nearby ("shadow") Hamiltonian:

$$\tilde{H} = H + \frac{h^2}{12} p^2 V''(q) - \frac{h^2}{24} V'(q)^2 + O(h^4)$$

- An exact Hamiltonian flow conserves its own Hamiltonian. So leapfrog conserves $\tilde{H}$, and $H = \tilde{H} + O(h^2)$ can oscillate but never drift.
- Measured on the 90° pendulum over 200 time units: at $h = 0.2$, H varies by 8.3×10^{-3} and $\tilde{H}$ by 7.0×10^{-5} ; at $h = 0.1$, by 2.1×10^{-3} and 4.4×10^{-6} .
- RK4 is not the exact flow of any Hamiltonian: no $\tilde{H}$ exists, and its error accumulates as the steady drift seen earlier.

The fine print



- **Fixed step.** Changing h from step to step (adaptive control) changes the shadow Hamiltonian as you go, and the bounded error is lost.
- **Hamiltonian systems that split.** Leapfrog needs $H = T(p) + V(q)$, two pieces each solved exactly. With friction or a driving force there is no energy to conserve; when H mixes q and p (a charged particle in a magnetic field), a different splitting is needed.
- **Second order.** Higher orders come from composing leapfrog steps: three substeps of sizes $w_1 h$, $w_0 h$, $w_1 h$ with $w_1 = 1/(2 - 2^{1/3}) \approx 1.35$ and $w_0 = 1 - 2w_1 \approx -1.70$ give a fourth-order symplectic method (Yoshida 1990).
- **Not more accurate.** Over a few periods RK4 wins, as the figure showed. Symplectic methods win over long runs.

Which method, when



the problem	a good default
general, non-stiff, accuracy requested	adaptive Dormand-Prince
non-stiff, smooth, f very expensive	Adams (predictor-corrector)
stiff	implicit: BDF
Hamiltonian, very long run	symplectic, fixed step: leapfrog and its compositions

Symplectic integrators run molecular dynamics codes, long-term planetary simulations, and particle pushers in plasma codes.